

Data Structure -

1. Linear
 - a. Array
 - b. Stack
 - c. Queue
 - d. Linked list
 - e. String
2. Non linear
 - a. Tree
 - i. Binary Tree (BT)
 - ii. Binary Search Tree (BST)
 - iii. Traversal of BT
 1. Inorder
 2. Preorder
 3. Postorder
 - b. Graph
 - i. Graph searching techniques -
 1. BFS
 2. DFS
 - ii. Shortest path finding techniques (Dijkstra)
 - iii. Overcoming the vulnerabilities of Dijkstra algorithm
 - iv. Enhanced Shortest path finding techniques
 - v. Minimum spanning tree
 1. Prim's algorithm
 2. Kruskal algorithm
 - c. Hash
 - d. Heap
 - i. Binary heap creation

Algorithms -

1. Sorting
 - a. Selection
 - b. Insertion
 - c. Bubble
 - d. Quick sort
 - e. Merge sort
2. Searching
 - a. Linear
 - b. Binary
3. Greedy
4. Divide and conquer
 - a. Quick sort
 - b. Merge sort
5. Dynamic programming
 - a. Coin exchange
 - b. 0/1 Knapsack
6. Hoffman coding
7. Misc

Array -

Stack -

Example - Convert the following infix expressions into its equivalent postfix expressions - $(A + B^D) / (E - F) + G$

Step	Infix part	Postfix
1	$B \wedge D$	$BD^{\wedge}$
2	$A + (B \wedge D)$	$ABD^{\wedge}+$
3	$E - F$	$EF-$
4	$(A + B^D) / (E - F)$	$ABD^{\wedge}+EF- /$
5	$\dots + G$	$ABD^{\wedge}+EF- /G+$

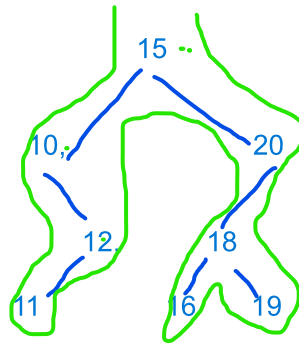
Binary Search Tree (BST) -

Example - Suppose the values 10, -4, 15, 30, 20, 5, 60, 19 are inserted in that order into an initially empty binary search tree. Now, draw the binary tree.

Tree traversal techniques -

Example - The **preorder traversal** of a binary search tree is - 15, 10, 12, 11, 20, 18, 16, 19. Which one of the following is the **postorder traversal** of the tree?

Post-order = 11, 12, 10, 16, 19, 18, 20, 15



Graph representation -

Feature	Adjacency Matrix	Adjacency List																																	
Example A / \ B---C D	<table><tr><td></td><td>A</td><td>B</td><td>C</td><td>D</td></tr><tr><td>A</td><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>B</td><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>C</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>D</td><td>0</td><td>1</td><td>0</td><td>0</td></tr></table>		A	B	C	D	A	0	1	1	0	B	1	0	1	1	C	1	1	0	0	D	0	1	0	0	<table><tr><td>A</td><td>B, C</td></tr><tr><td>B</td><td>A, C, D</td></tr><tr><td>C</td><td>A, B</td></tr><tr><td>D</td><td>B</td></tr></table>	A	B, C	B	A, C, D	C	A, B	D	B
	A	B	C	D																															
A	0	1	1	0																															
B	1	0	1	1																															
C	1	1	0	0																															
D	0	1	0	0																															
A	B, C																																		
B	A, C, D																																		
C	A, B																																		
D	B																																		

Graph traversal algorithms -

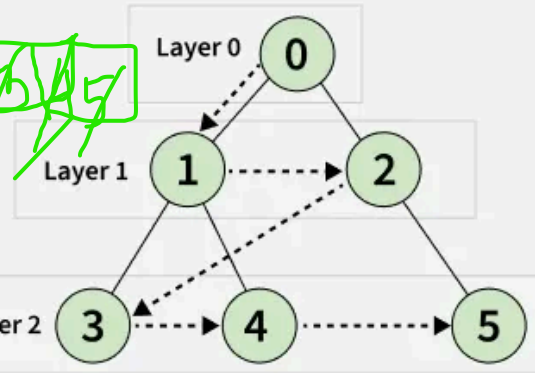
- queue
- first explore neighbours of a node

BFS

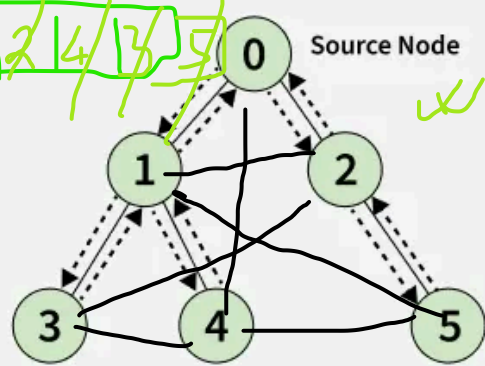
- stack
- explore the depth of a node

DFS

VS



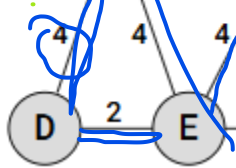
Output: 0, 1, 2, 3, 4, 5



Output: 0, 1, 3, 4, 2, 5

6, 1, 2, 3, 4, 5
2, 1

✓ Dijkstra alg



Algorithm finished

Dynamic Programming -

Discuss 0/1 Knapsack with an example

Find the optimal solution for the 0/1 knapsack problem making use of a dynamic programming approach. Consider- $n = 4$, $w = 5$ kg, weights $(w_1, w_2, w_3, w_4) = (2, 3, 4, 5)$ and benefits $(b_1, b_2, b_3, b_4) = (3, 4, 5, 6)$. Draw a table say 'T' with $(n+1) = 4 + 1 = 5$ number of rows and $(w+1) = 5 + 1 = 6$ number of columns. Fill all the boxes of 0th row and 0th column with 0. Start filling the table row wise top to bottom from left to right using the formula-

$$\text{If } w_i > w, \text{ then } K[i-1][w]$$
$$\text{Otherwise, } K[i][w] = \max \{ K[i-1][w] \text{ or } b_i + K[i-1][w - w_i] \}$$

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

Item (i)	Weight (w _i)	Benefit (b _i)
1	2	3
2	3	4
3	4	5
4	5	6

The last entry represents the maximum possible value that can be put into the knapsack. So, maximum possible value that can be put into the knapsack = 7.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7
4	0	0	3	4	5	7

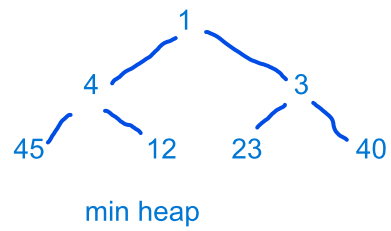
Heap -

Create min heap from given array

[12, 4, 23, 45, 1, 3, 40]

max heap =

min heap =



Hash -

1. **Static Hashing**

To solve this issue, we can use either **Overflow Chaining** or **Linear Probing**.

a) **Overflow Chaining**

b) **Linear Probing**

2. **Dynamic Hashing**

15, 18, 23, 54, 56

$h(k) = k \% 10 =$

44, 33, 96

.

0

1

2

3 - 23 -> 33

4 - 54 -> 44 -> 104

5 - 15

6 - 56 -> 96

7 -

8 - 18

9 -

Exercise 01 - A file contains the following characters with the frequencies as shown.

Character	Frequency
a	10
e	15
i	12
o	3
u	4
s	13
t	1

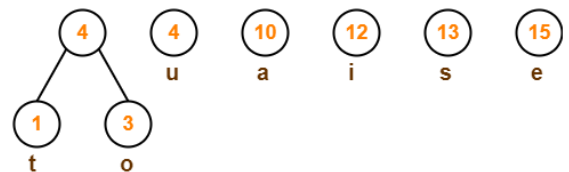
If Huffman Coding is used for data compression, determine-

1. Huffman Code for each character
2. Average code length
3. Length of Huffman encoded message (in bits)

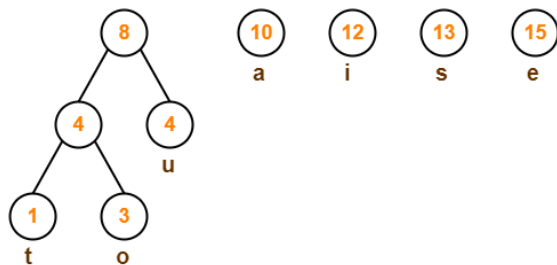
Step-01:



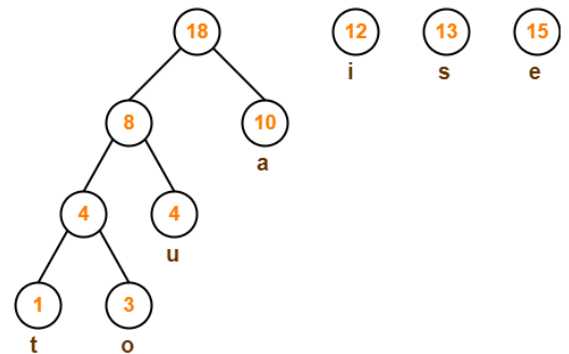
Step-02:



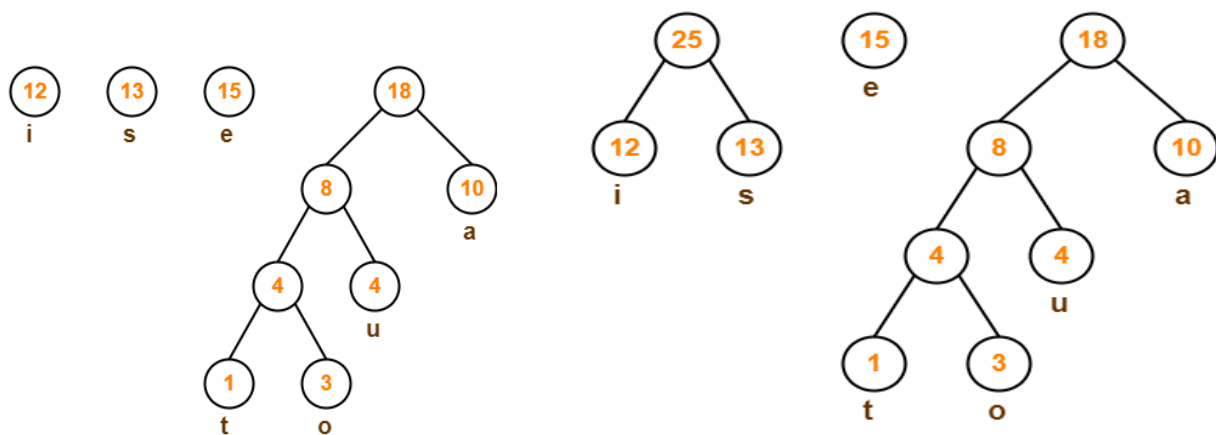
Step-03:



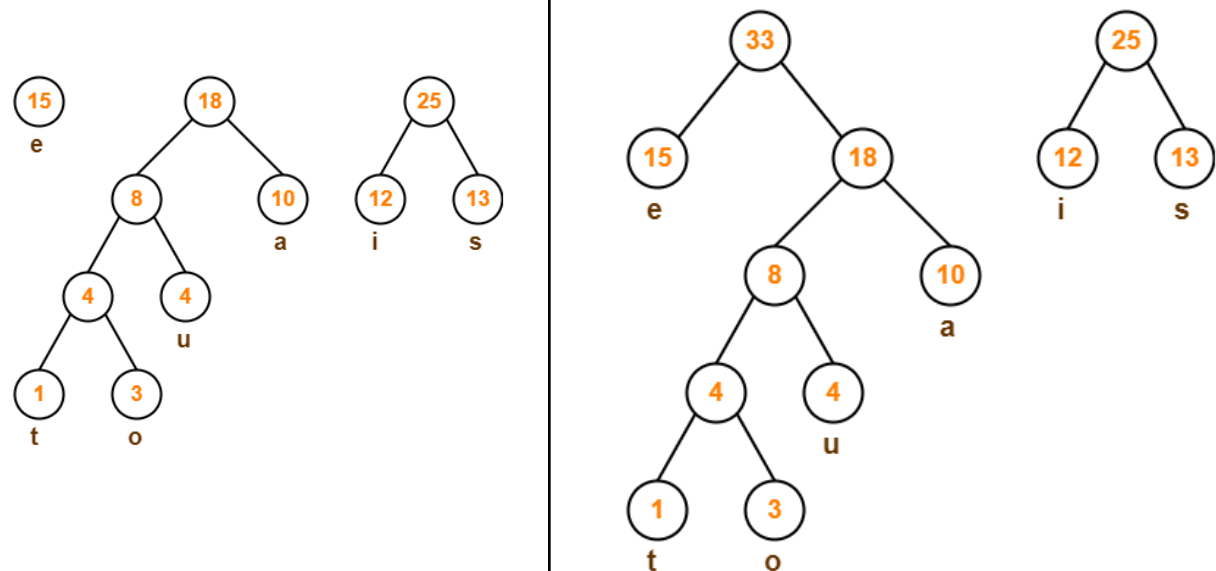
Step-04:



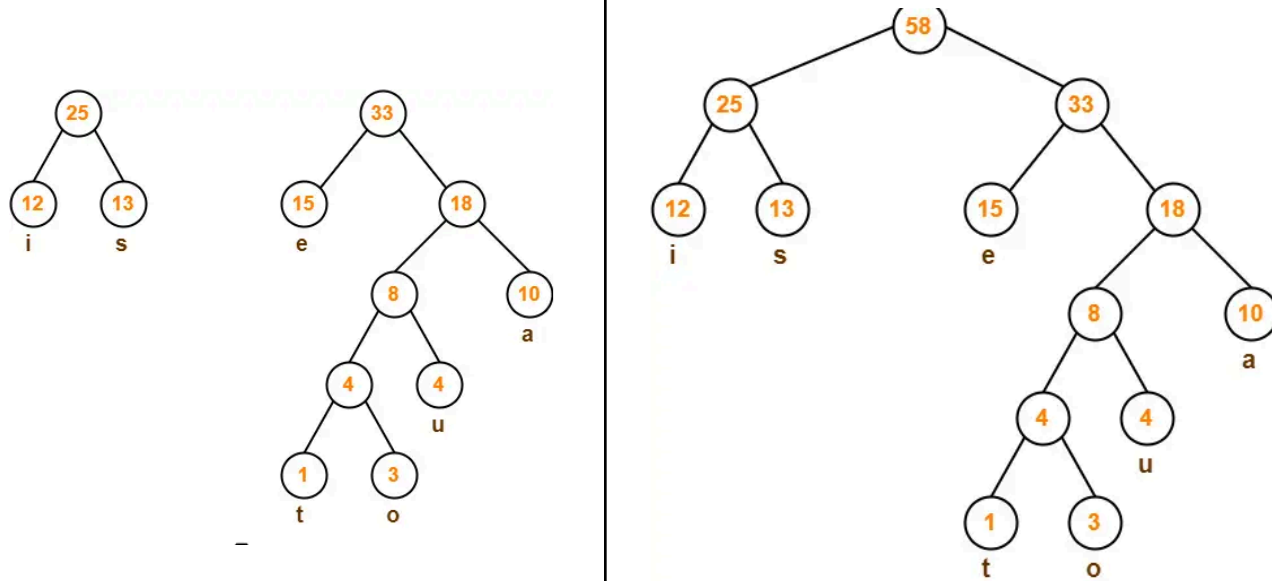
Step-05:

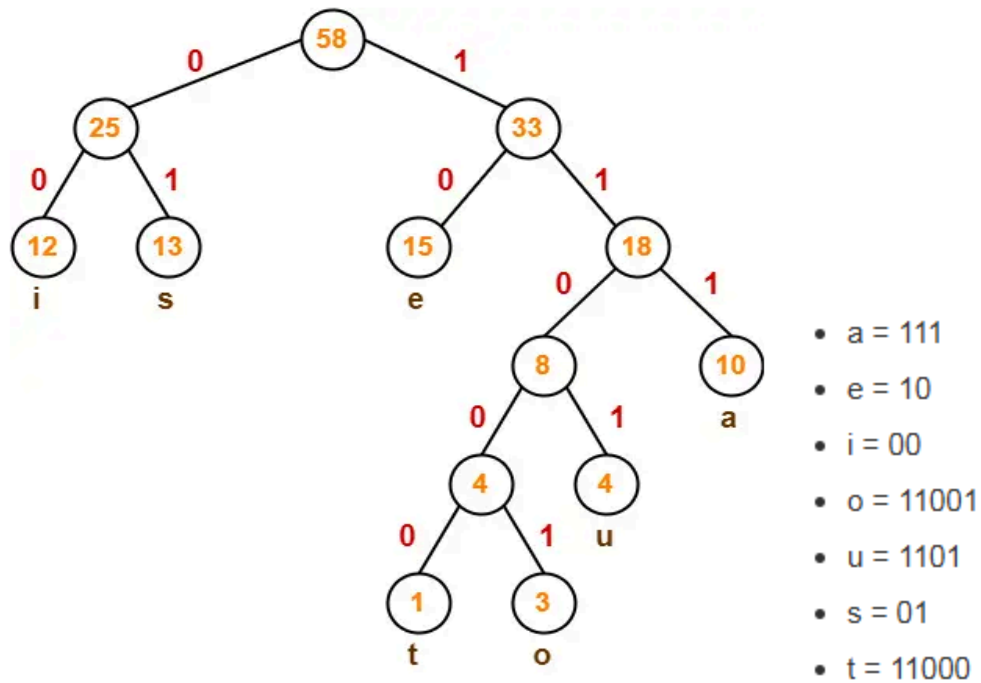


Step-06:



Step-07:





Average code length = $\sum (\text{frequency}_i \times \text{code length}_i) / \sum (\text{frequency}_i)$
 = $\{ (10 \times 3) + (15 \times 2) + (12 \times 2) + (3 \times 5) + (4 \times 4) + (13 \times 2) + (1 \times 5) \} / (10 + 15 + 12 + 3 + 4 + 13 + 1) = 2.52$

Total number of bits in Huffman encoded message
 = Total number of characters in the message x Average code length per character
 = $58 \times 2.52 = 146.16 \approx 147$ bits